



CS 610

GPU Development Tools

Jieyang Chen



UNIVERSITY OF
OREGON

Computer Science

CS 610 GPU Programming, Winter 2026

CUDA Compute Capabilities



UNIVERSITY OF
OREGON

Computer Science

CS 610 GPU Programming, Winter 2026

CUDA Compute Capability (CC)

- The compute capabilities refer to specified sets of **hardware** features present on the different generations of NVIDIA GPUs.
- Also known as SM version
- Format: XY
 - X = major version (core architecture)
 - Y = minor version (incremental improvement to the core architecture)

CUDA Compute Capability (CC)

Generation	Tesla	Fermi	Kepler	Maxwell	Pascal	Volta	Turing	Ampere	Ada Lovelace	Hopper	Blackwell
CC	1.0-1.3	2.0-2.1	3.0-3.7	5.0	6.0	7.0-7.2	7.5	8.0-8.7	8.9	9.0	10.0

<https://developer.nvidia.com/cuda/gpus>

CUDA Compute Capability (CC)

- <https://en.wikipedia.org/wiki/CUDA>

Feature support (unlisted features are supported for all compute capabilities)	Compute capability (version)										
	1.0, 1.1	1.2, 1.3	2.x	3.0	3.2	3.5, 3.7, 5.x, 6.x, 7.0, 7.2	7.5	8.x	9.0		
Warp vote functions (<code>__all()</code> , <code>__any()</code>)	No	Yes									
Warp vote functions (<code>__ballot()</code>)	No	Yes									
Memory fence functions (<code>__threadfence_system()</code>)											
Synchronization functions (<code>__syncthreads_count()</code> , <code>__syncthreads_and()</code> , <code>__syncthreads_or()</code>)											
Surface functions											
3D grid of thread blocks	No	Yes									
Warp shuffle functions											
Unified memory programming	No			Yes							
Funnel shift	No				Yes						
Dynamic parallelism	No					Yes					
Uniform Datapath ^[53]	No								Yes		
Hardware-accelerated async-copy	No								Yes		
Hardware-accelerated <i>split arrive/wait barrier</i>											
Warp-level support for reduction ops											
L2 cache residency management											
DPX instructions for accelerated dynamic programming	No								Yes		
Distributed shared memory											
Thread block cluster											
Tensor memory accelerator (TMA) unit											
Feature support (unlisted features are supported for all compute capabilities)	Compute capability (version)										
	1.0,1.1	1.2,1.3	2.x	3.0	3.2	3.5, 3.7, 5.x, 6.x, 7.0, 7.2	7.5	8.x	9.0		

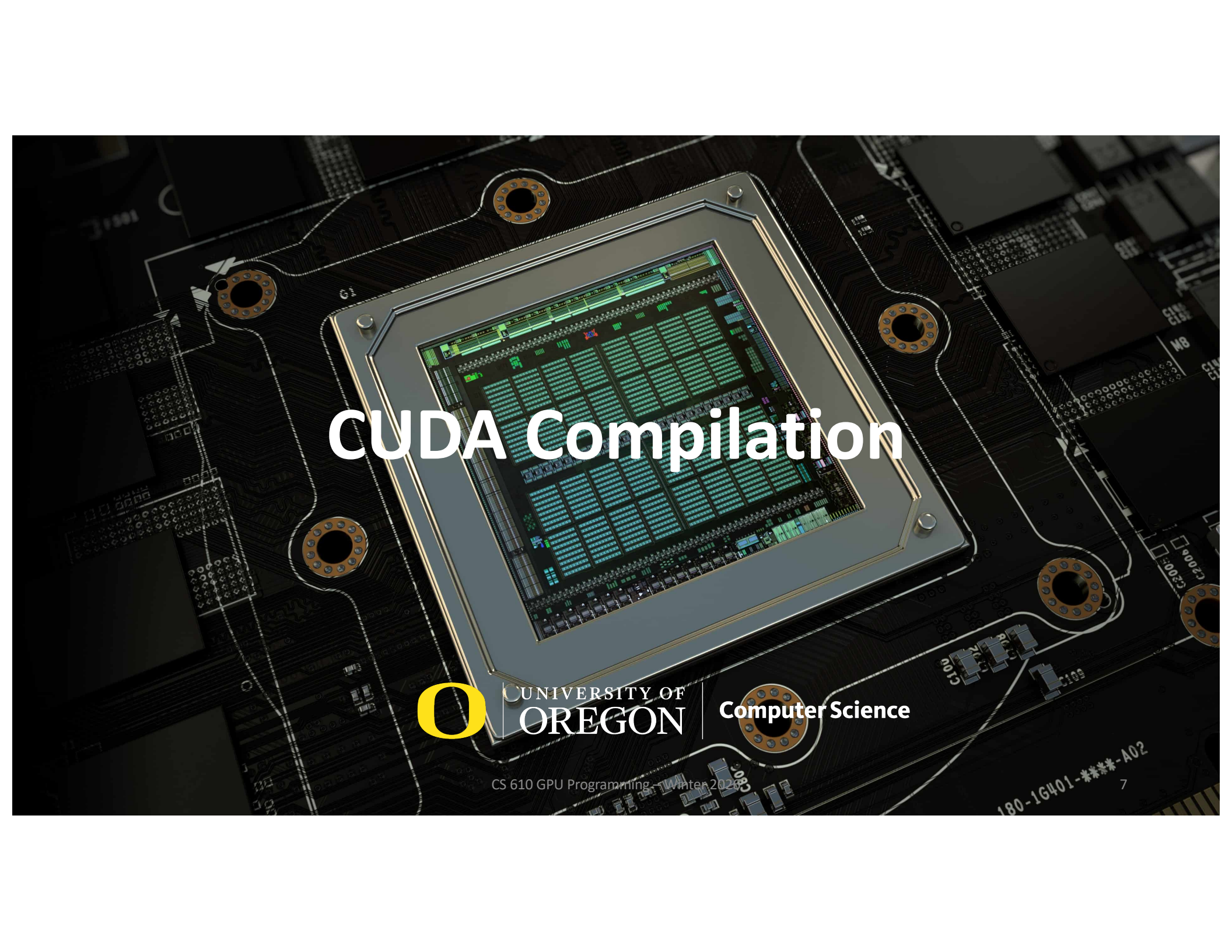


CUDA Compute Capability (CC)

- Do not confuse CUDA Compute Capability with CUDA Version
- CUDA Compute Capability = hardware architecture version
- CUDA Version (CUDA SDK version) = software version

CUDA SDK Version(s)	Tesla	Fermi	Kepler (Early)	Kepler (Late)	Maxwell	Pascal	Volta	Turing	Ampere	Ada Lovelace	Hopper	Blackwell
1.0 ^[35]	1.0 – 1.1											
1.1	1.0 – 1.1+x											
2.0	1.0 – 1.1+x											
2.1 – 2.3.1 ^{[36][37][38][39]}	1.0 – 1.3											
3.0 – 3.1 ^{[40][41]}	1.0	2.0										
3.2 ^[42]	1.0	2.1										
4.0 – 4.2	1.0	2.1										
5.0 – 5.5	1.0			3.5								
6.0	1.0		3.2	3.5								
6.5	1.1			3.7	5.x							
7.0 – 7.5		2.0			5.x							
8.0		2.0				6.x						
9.0 – 9.2			3.0				7.0 – 7.2					
10.0 – 10.2			3.0					7.5				
11.0 ^[43]				3.5					8.0			
11.1 – 11.4 ^[44]				3.5					8.6			
11.5 – 11.7.1 ^[45]				3.5					8.7			
11.8 ^[46]				3.5						8.9	9.0	
12.0 – 12.4					5.0						9.0	

Older CCs are no longer supported by newer SDKs



CUDA Compilation



UNIVERSITY OF
OREGON

Computer Science

CS 610 GPU Programming, Winter 2026

Compiling for GPU Code



CS 610 GPU Programming: Winter 2026

Compilation

- Compilation is the process of translating source code written in a high-level programming language into a lower-level language, such as machine code, that can be executed by a computer.



Binary Compatibility

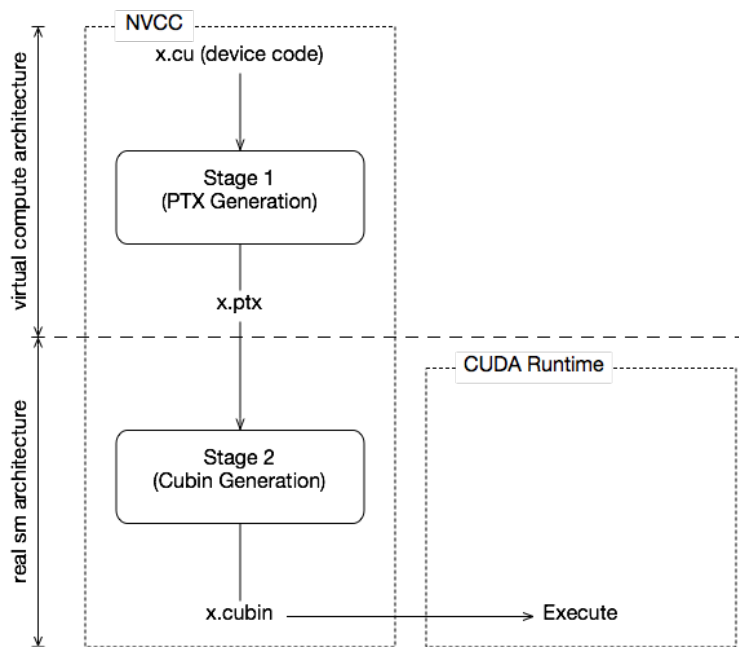
- **Binary Compatible:** Binary executable compiled on one system can run on another system without recompilation.
 - Binary compatible is commonly achieved on CPUs through shared instruction set such as x86
- **Cross-vendor:** Most binaries compiled for x86 will work on x86 CPUs across different major vendors (Intel, AMD)
- **Forward-compatible:** Most binaries compiled for older x86 instruction set will work on newer versions of the CPU

Binary Compatibility

- GPU architectures evolve much faster than CPU architectures
- New GPU generations introduce major improvements in functionality and/or chip architecture.
- Binary compatibility of GPU applications across different generations **cannot be guaranteed. No forward or backward compatibility**
 - GPU programs compiled for Ampere cannot run on Volta
- Binary compatibility within one GPU generation **can be guaranteed**
 - GPU program compiled for consumer-class Ampere GPUs can run on server-class Ampere GPUs

Two stage compilation model

- Solving GPU compatibility: **two stage compilation model**



Two-level architectures

- **Virtual architecture**: the set of hardware capabilities and features required by CUDA program
- **Real architecture**: the *real* GPU architecture that the CUDA program intend to execute on



Two stage compilation model

Virtual architecture list

- Format: `compute_<compute capability>`

<code>compute_50</code> , <code>compute_52</code> , and <code>compute_53</code>	Maxwell support
<code>compute_60</code> , <code>compute_61</code> , and <code>compute_62</code>	Pascal support
<code>compute_70</code> and <code>compute_72</code>	Volta support
<code>compute_75</code>	Turing support
<code>compute_80</code> , <code>compute_86</code> and <code>compute_87</code>	NVIDIA Ampere GPU architecture support
<code>compute_89</code>	Ada support
<code>compute_90</code> , <code>compute_90a</code>	Hopper support



Two stage compilation model

Real architecture list

- Format: sm_<compute capability>

sm_50, sm_52 and sm_53	Maxwell support
sm_60, sm_61, and sm_62	Pascal support
sm_70 and sm_72	Volta support
sm_75	Turing support
sm_80, sm_86 and sm_87	NVIDIA Ampere GPU architecture support
sm_89	Ada support
sm_90, sm_90a	Hopper support

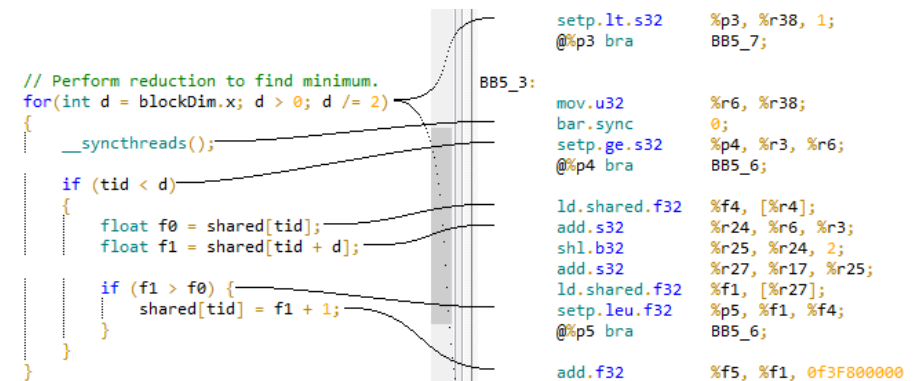


Two stage compilation model

- Solving GPU compatibility: **two stage compilation model**

Compilation Stage 1: CUDA code → PTX code

- PTX code is an intermediate representation code generated by NVCC
- PTX code generated for a chosen virtual architecture
- PTX is forward compatible but not backward compatible



Two stage compilation model

- Solving GPU compatibility: **two stage compilation model**

Compilation Stage 2: PTX code → Binary

- Generate the final binary executable
- Binary is generated for a chosen real architecture
- The *real* architecture must be an **implementation** of the *virtual* architecture.
- Binary executable is **not forward or backward compatible**

```
setp.lt.s32    %p3, %r38, 1;
@%p3 bra      BB5_7;

mov.u32        %r6, %r38;
bar.sync       0;
setp.ge.s32    %p4, %r3, %r6;
@%p4 bra      BB5_6;

ld.shared.f32  %f4, [%r4];
add.s32        %r24, %r6, %r3;
shl.b32        %r25, %r24, 2;
add.s32        %r27, %r17, %r25;
ld.shared.f32  %f1, [%r27];
setp.leu.f32   %p5, %f1, %f4;
@%p5 bra      BB5_6;

add.f32        %f5, %f1, 0f3F800000
```



```
00000000 0000 0001 0001 1010 0010 0001 0004 0128
00000010 0000 0016 0000 0028 0000 0018 0000 0020
00000020 0000 0001 0004 0000 0000 0000 0000 0000
00000030 0000 0000 0000 0010 0000 0000 0000 0204
00000040 0004 3384 0004 c7c8 00c8 4748 0048 a8e0
00000050 00e9 6a69 0069 a8a9 00a9 2828 0028 fdfc
00000060 00fc 1819 0019 9898 0098 d9d8 00d8 5857
00000070 0057 7b7a 007a bab9 00b9 3a3c 003c 8888
00000080 8888 8888 8888 8888 288e be88 8888 8888
00000090 3b83 5788 8888 8888 7667 778e 8828 8888
000000a0 d01f 7abd 8818 8888 467c 585f 8814 8188
000000b0 8b86 e8f7 88aa 8388 8b3b 88f3 88bd e988
000000c0 8a18 886c e841 c988 b328 6871 688e 958b
000000d0 a948 5862 5884 7e81 3788 1ab4 5a84 3eec
000000e0 3d86 dcb8 5cbb 8888 8888 8888 8888 8888
000000f0 8888 8888 8888 8888 8888 8888 8888 0000
00001000 0000 0000 0000 0000 0000 0000 0000 0000
*
00001300 0000 0000 0000 0000 0000 0000 0000
000013e
```

Two stage compilation model

What does choosing a virtual architecture implies?

- Virtual architecture is more of a statement on the GPU capabilities that the application requires
 - If you need to use tensor cores, then you need `>=compute_7.0`
 - If you need to use FP64 atomicAdd, then you need `>= compute_6.0`
- PTX code generated for a virtual architecture is **forward compatible** not backward compatible
 - PTX for an older virtual arch. (e.g., `compute_50`) **can** be compiled to a binary of a newer real arch. (e.g., `sm_75`).
 - PTX for a newer virtual arch. (e.g., `compute_80`) **cannot** be compiled to a binary of an older real arch. (e.g., `sm_50`).

Two stage compilation model

- A *smaller* virtual architecture allows a *wider* range of real architectures for the second compilation stage.
- Conversely, specifying a virtual architecture that provides features *unused* by the CUDA program *unnecessarily restricts* the set of possible real architecture that can be specified in the second compilation stage

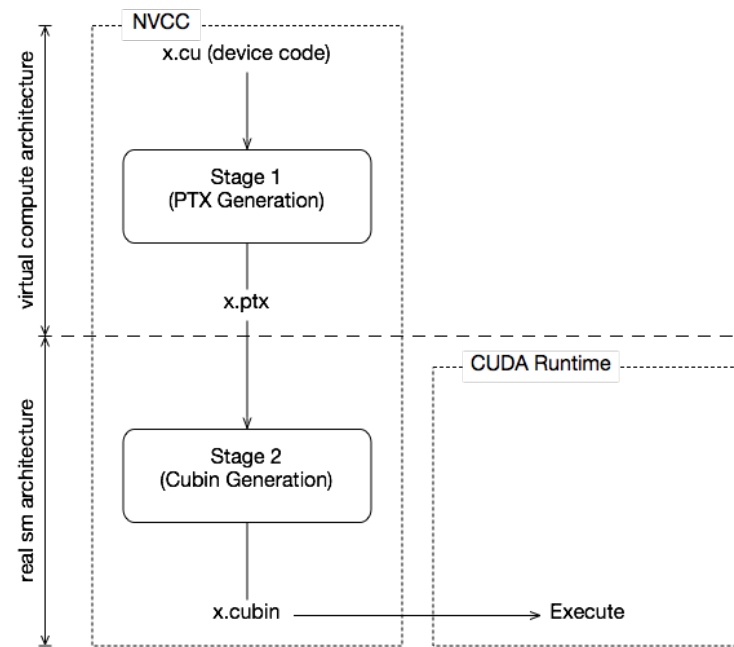
Two stage compilation model

How to choose virtual architecture and real architecture to maximize compatibility?

- Virtual architecture should always be chosen *as low as possible*, thereby maximizing the actual GPUs to run on
- Real architecture should be chosen *as high as possible* to be close to the actual GPU for better compilation optimization

Two stage compilation model

- Solving GPU compatibility: **two stage compilation model**



Config NVCC

```
nvcc <source code> --gpu-architecture=<virtual> --gpu-code=<real>
```

Example:

```
nvcc <source code> --gpu-architecture=compute_70 --gpu-code=sm_70
```

Virtual architecture: Volta

Real architecture: Volta

Same architecture

Config NVCC

```
nvcc <source code> --gpu-architecture=<virtual> --gpu-code=<real>
```

Example:

```
nvcc <source code> --gpu-architecture=compute_50 --gpu-code=sm_80
```

Virtual architecture: Maxwell

Real architecture: Ampere

Virtual architecture is forward compatible

Config NVCC

```
nvcc <source code> --gpu-architecture=<virtual> --gpu-code=<real>
```

Example:

```
nvcc <source code> --gpu-architecture=compute_70 --gpu-code=sm_50
```

Virtual architecture: Volta

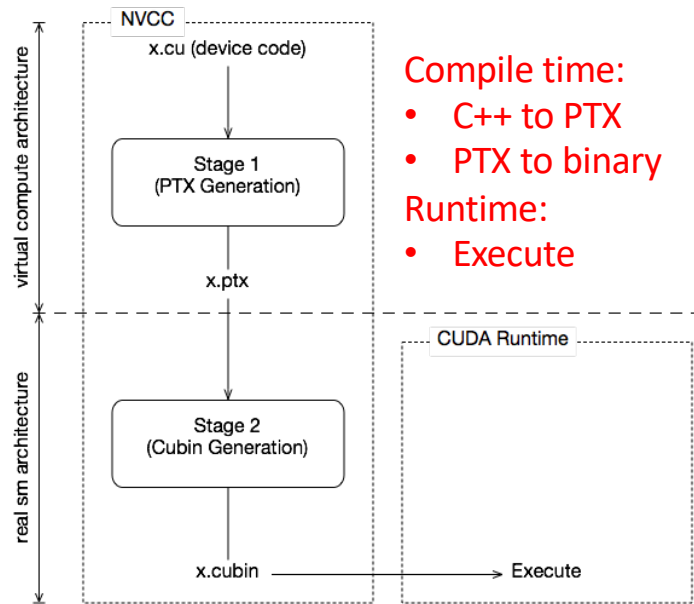
Real architecture: Maxwell

Error: virtual architecture is not back compatible

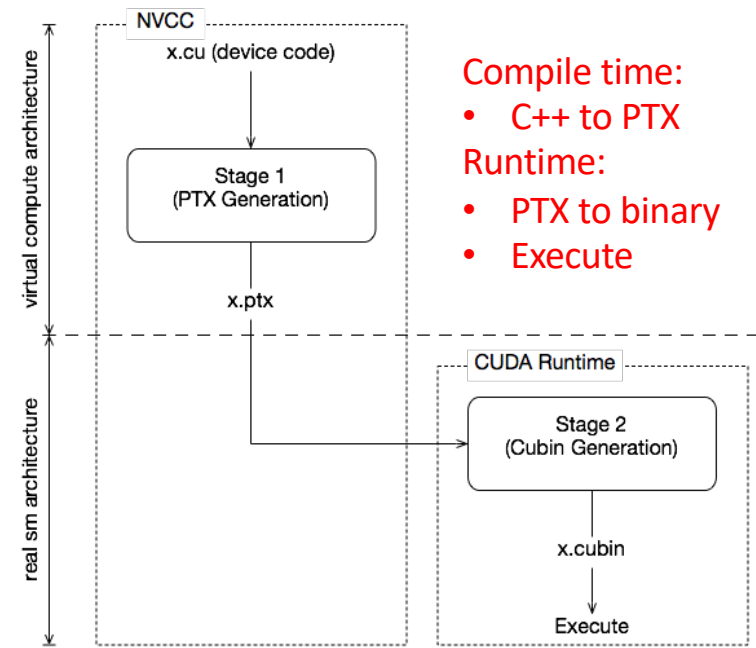
Two stage compilation model

What if the real GPU architecture is not known at compilation time?

- Just-in-Time (JIT) compilation



Two-Stage Compilation



Just-in-Time Compilation



Config NVCC for JIT compilation

```
nvcc <source code> --gpu-architecture=<virtual> --gpu-code=<virtual>
```

Second compilation stage is skipped, only PTX code is generated

PTX code will be compiled to binary at runtime when real GPU architecture is known

Forward compatible: Can be executed on all same or later architectures

Example:

```
nvcc <source code> --gpu-architecture=compute_70 --gpu-code=compute_70
```

Virtual architecture: Volta

Real architecture: skipped

Can execute on $\geq$ sm_70

Config NVCC for JIT compilation

```
nvcc <source code> --gpu-architecture=<virtual> --gpu-code=<virtual>
```

Second compilation stage is skipped, only PTX code is generated

PTX code will be compiled to binary at runtime when real GPU architecture is known

Forward compatible: Can be executed on all same or later architectures

Example:

```
nvcc <source code> --gpu-architecture=compute_50 --gpu-code=compute_50
```

Virtual architecture: Maxwell

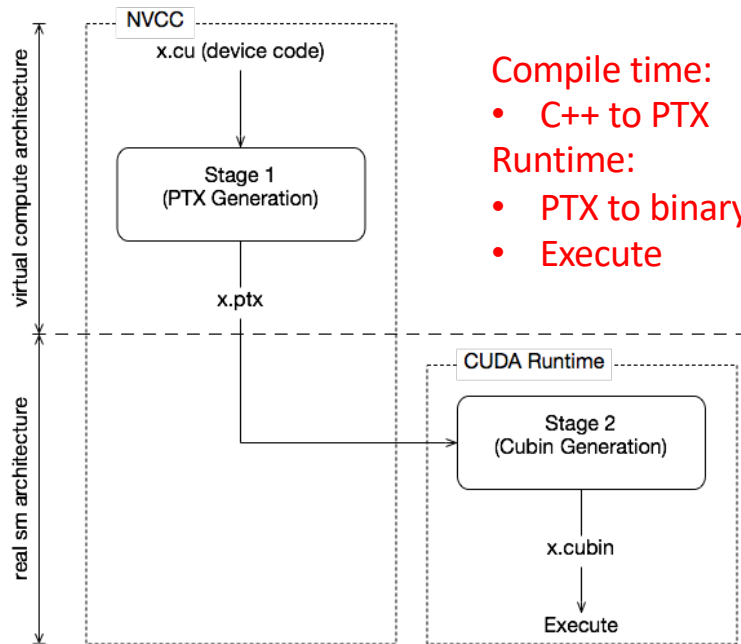
Real architecture: skipped

Can execute on $\geq$ sm_50

Two stage compilation model

What if the real GPU architecture is not known at compilation time?

- Just-in-Time (JIT) compilation



JIT compilation can improve compatibility
But it can increase application startup delay

What is causing delay?



Two stage compilation model

What if we know some of the target architectures but we also want to maintain compatibility on potential unknown architectures?

- Fatbinary (Fatbin): packing **multiple architecture generations** of **PTX** and **executable binary** in one single binary file
 - Executable binary : ensure delay-free execution on known architectures
 - PTX: ensure compatibility on potential unknown architectures

Config NVCC for Fatbinary

```
nvcc <source code> --gpu-architecture=<virtual> --gpu-code=<virtual>, <real>,<real>...
```

Example:

```
nvcc <source code> --gpu-architecture=compute_50 --gpu-code=compute_50, sm_70, sm_80
```

Virtual architecture: Maxwell

Real architecture: Maxwell (PTX), Volta(executable), Ampere(Executable)

Can execute on sm_50, sm_70, sm_80 zero startup delay

Can execute on >=sm_50 with JIT compilation



Two stage compilation model

Previously, we say virtual architecture should always be chosen **as low as possible**, thereby maximizing the actual GPUs to run on

What if we want to use new HW features in newer architectures but we also want to maintain compatibility with older architectures?

- Virtual Architecture Macros: let source code distinguish virtual architecture being build
- Fatbinary: packing **multiple architecture generations** of **executable code** in one single binary file

Config NVCC for Fatbinary with multi-virtual arch

```
nvcc <source code> --generate-code arch=<virtual1>,code=<real1> \  
                    --generate-code arch=<virtual2>,code=<real2> \  
                    --generate-code arch=<virtual3>,code=<real3> \ ....
```

Example:

```
nvcc <source code> \  
    --generate-code arch=compute_80,code=sm_80 \  
    --generate-code arch=compute_70,code=sm_70 \  
    --generate-code arch=compute_50,code=sm_50
```

Virtual architecture: Maxwell, Volta, Ampere

Real architecture: Maxwell, Volta, Ampere



Virtual Architecture Macros

`__CUDA_ARCH__ = XY0`

```
nvcc <source code> \  
--generate-code arch=compute_80,code=sm_80 \  
--generate-code arch=compute_70,code=sm_70 \  
--generate-code arch=compute_50,code=sm_50
```

Compiler will iteratively compile for each arch
Each time `__CUDA_ARCH__` will equal to a certain arch version number

```
<source code>  
__device__ void some_device_fun(...parameters...) {  
    #if (__CUDA_ARCH__ == 500)  
        // execute following if running on sm_50  
        ....  
    #elif (__CUDA_ARCH__ == 700)  
        // execute following if running on sm_70  
        ....  
    #elif (__CUDA_ARCH__ == 800)  
        // execute following if running on sm_80  
        ....  
    #else  
        // execute following if running on other arch  
        ....  
    #endif  
}
```

Virtual Architecture Macros

`__CUDA_ARCH__ = XY0`

```
nvcc <source code> \  
--generate-code arch=compute_80,code=sm_80 \  
--generate-code arch=compute_70,code=sm_70 \  
--generate-code arch=compute_50,code=sm_50
```

Only this part is visible
to NVCC at current pass

Compiler will iteratively compile for each arch

Each time `__CUDA_ARCH__` will equal to a certain arch version number

1st pass of compilation

```
<source code>:  
__device__ void  
some_device_fun(...parameters...) {  
    #if (__CUDA_ARCH__ == 500)  
        // execute following if running on sm_50  
        ....  
    #elif (__CUDA_ARCH__ == 700)  
        // execute following if running on sm_70  
        ....  
    #elif (__CUDA_ARCH__ == 800)  
        // execute following if running on sm_80  
        ....  
    #endif  
}
```


Virtual Architecture Macros

`__CUDA_ARCH__ = XY0`

```
nvcc <source code> \  
--generate-code arch=compute_80,code=sm_80 \  
--generate-code arch=compute_70,code=sm_70 \  
--generate-code arch=compute_50,code=sm_50
```

Only this part is visible
to NVCC at current pass

Compiler will iteratively compile for each arch
Each time `__CUDA_ARCH__` will equal to a certain arch version number

2nd pass of compilation

```
<source code>:  
__device__ void  
some_device_fun(...parameters...) {  
    #if (__CUDA_ARCH__ == 500)  
        // execute following if running on sm_50  
        ....  
    #elif (__CUDA_ARCH__ == 700)  
        // execute following if running on sm_70  
        ....  
    #elif (__CUDA_ARCH__ == 800)  
        // execute following if running on sm_80  
        ....  
    #endif  
}
```

Virtual Architecture Macros

`__CUDA_ARCH__ = XY0`

Only this part is visible
to NVCC at current pass

```
nvcc <source code> \  
--generate-code arch=compute_80,code=sm_80 \  
--generate-code arch=compute_70,code=sm_70 \  
--generate-code arch=compute_50,code=sm_50
```

Compiler will iteratively compile for each arch
Each time `__CUDA_ARCH__` will equal to a certain arch version number

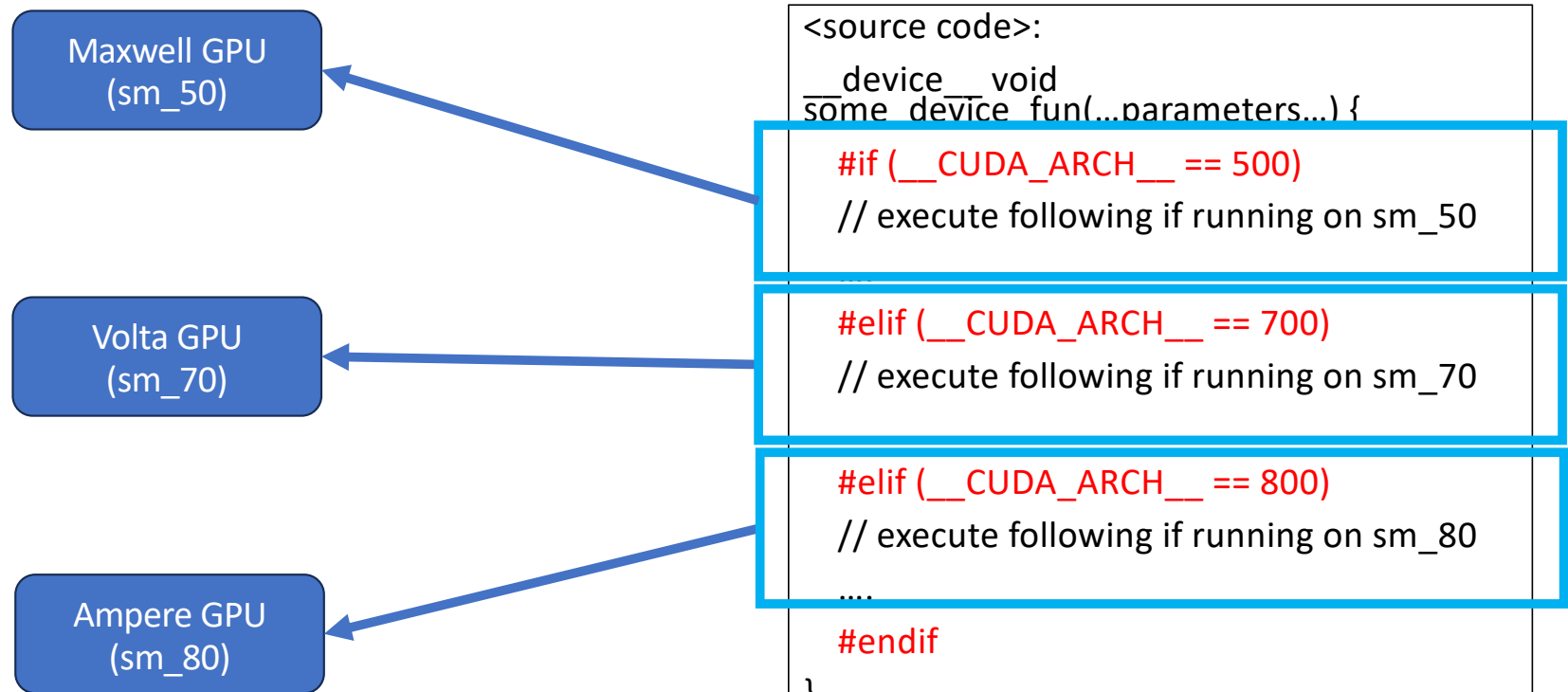
3rd pass of compilation

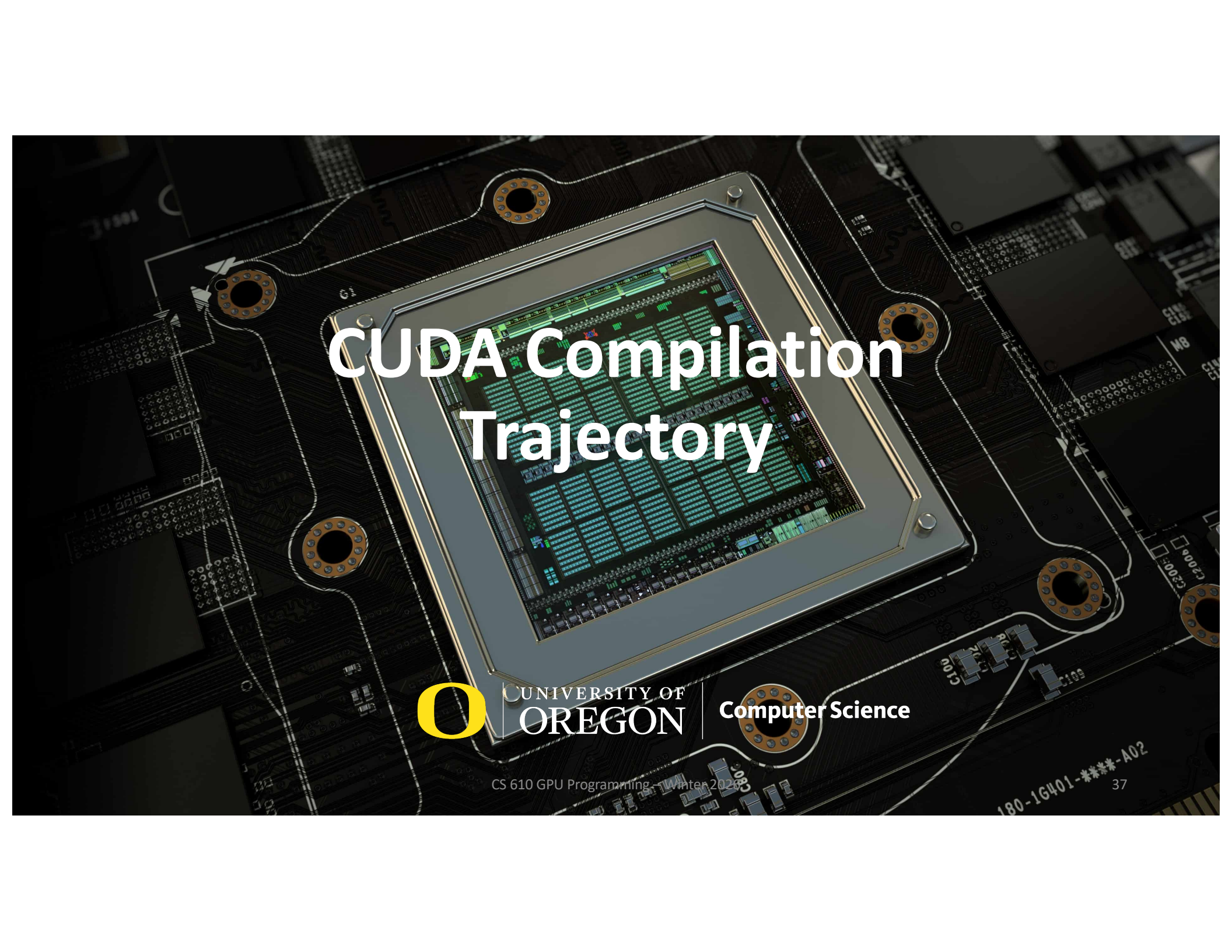
<source code>:

```
__device__ void  
some_device_fun(...parameters...) {  
    #if (__CUDA_ARCH__ == 500)  
        // execute following if running on sm_50  
  
    #elif (__CUDA_ARCH__ == 700)  
        // execute following if running on sm_70  
        ....  
    #elif (__CUDA_ARCH__ == 800)  
        // execute following if running on sm_80  
        ....  
    #endif  
}
```

Virtual Architecture Macros

Runtime polymorphism





CUDA Compilation Trajectory



UNIVERSITY OF
OREGON

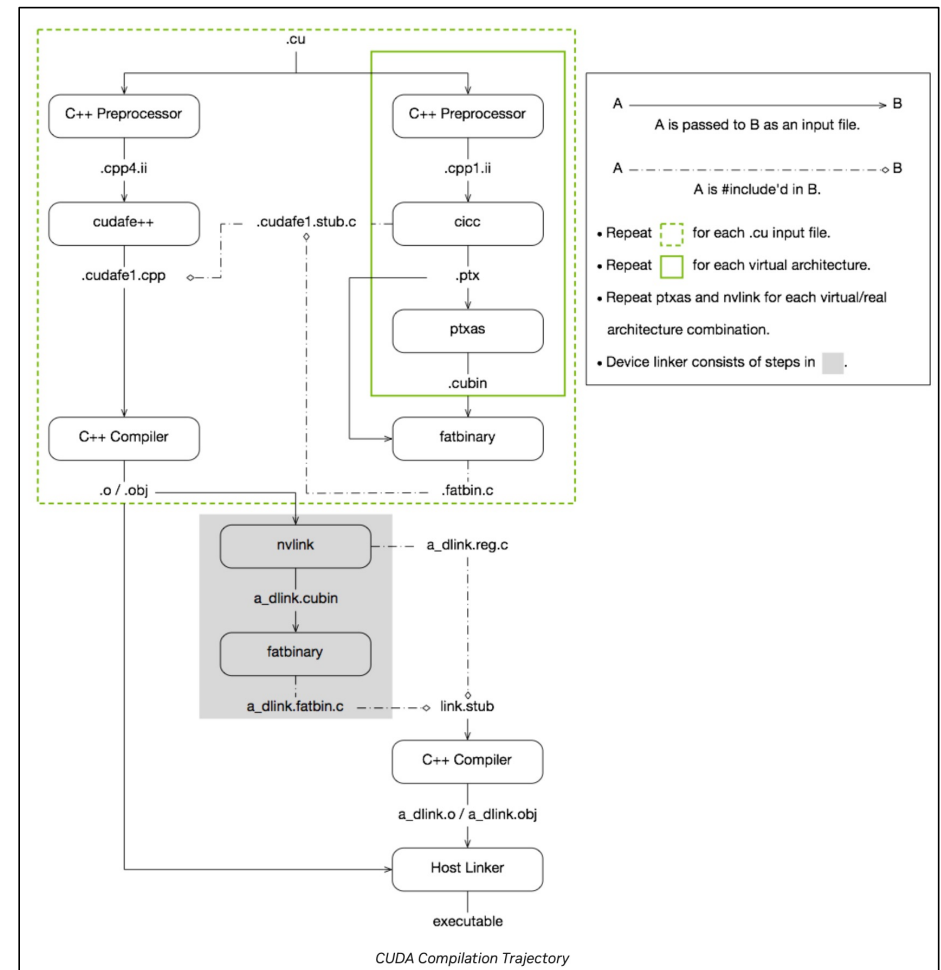
Computer Science

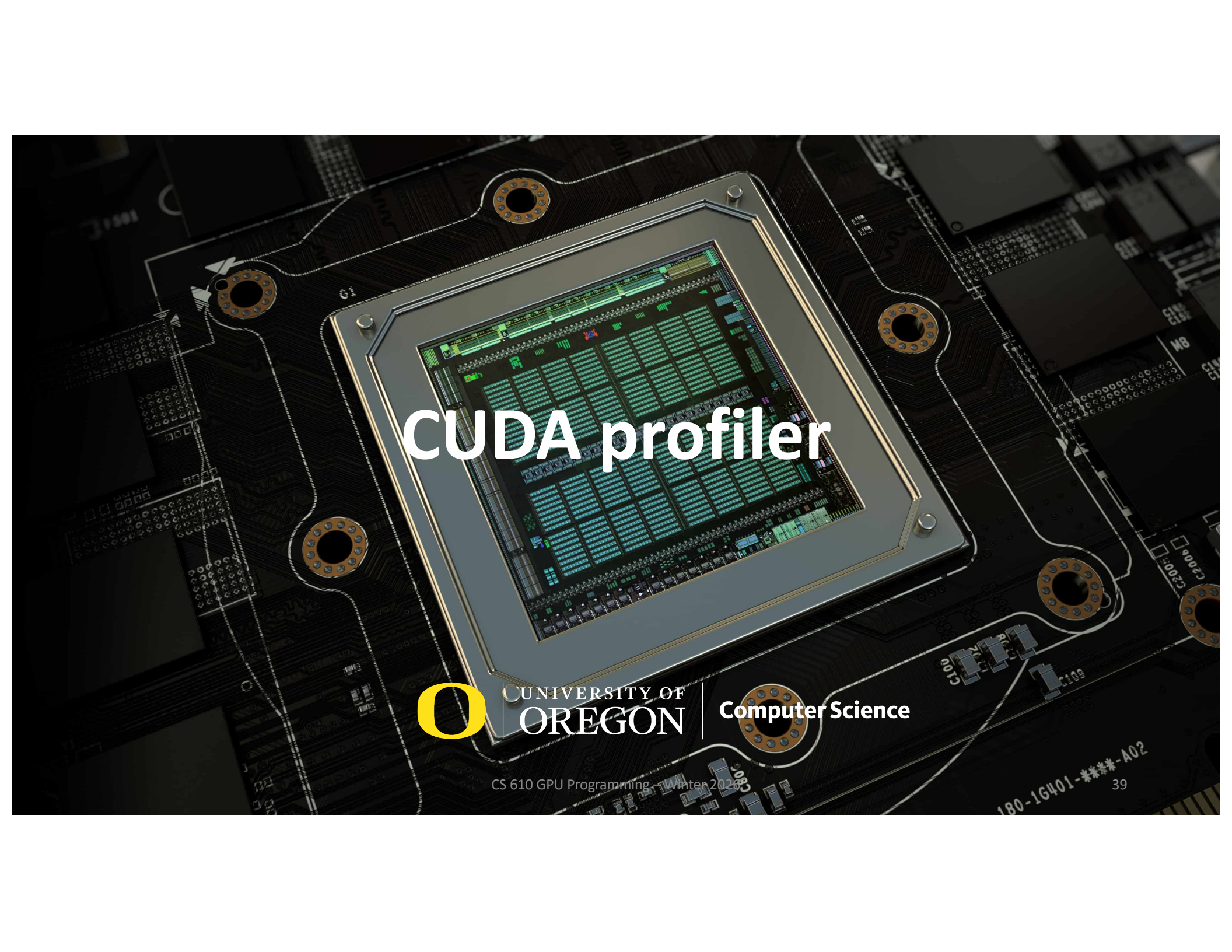
CS 610 GPU Programming, Winter 2026

37

Two Trajectory Model

1. NVCC compile device code to cubin
2. GCC compile host code to obj
3. cubin and obj are linked together





CUDA profiler



UNIVERSITY OF
OREGON

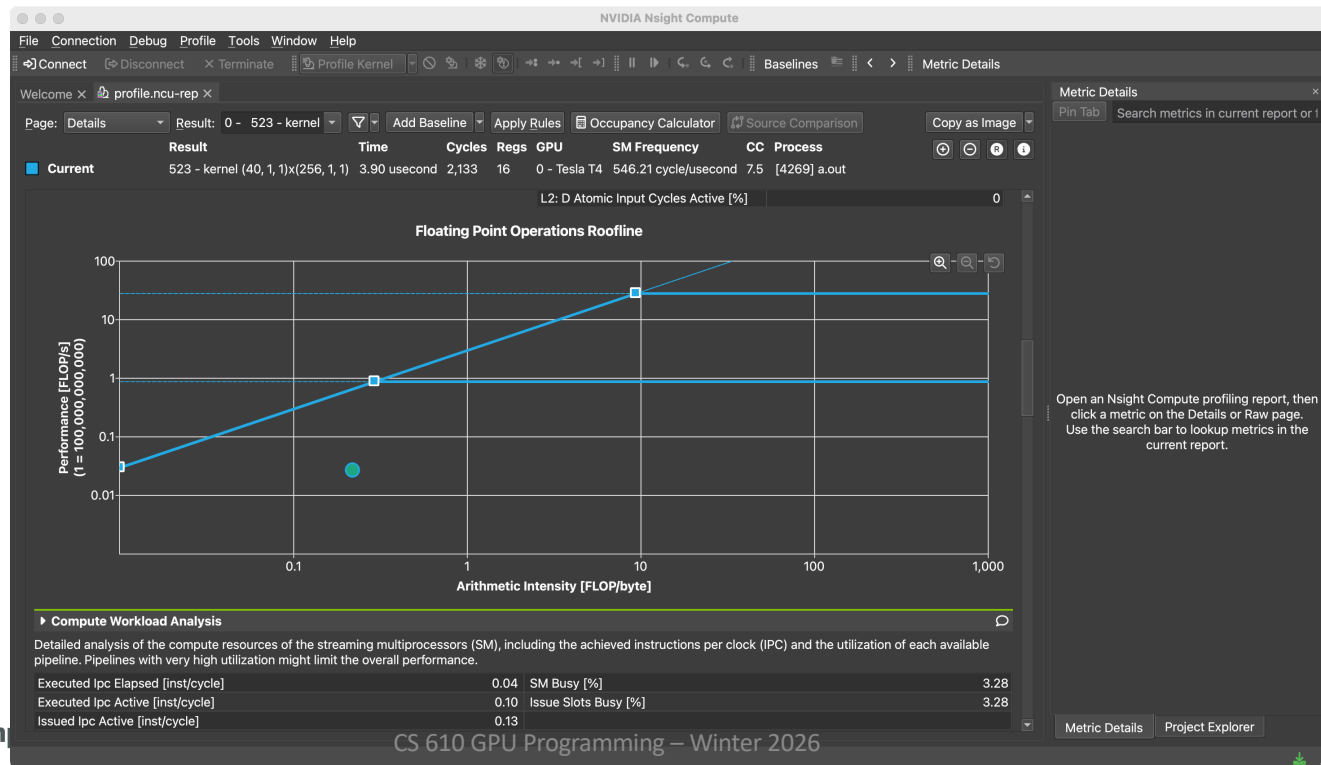
Computer Science

CS 610 GPU Programming, Winter 2026

39

Profile kernel

- Use Nvidia profiler: Nsight Compute
- On the server, profile with: `ncu -o profile --set full ./a.out`
- Open profile.ncu-rep using the Nsight Compute on your computer



Profile program

- Use Nvidia profiler: Nsight System
- On the server, profile with: `nsys profile -o report ./a.out`
- Open report.nsys-rep using the Nsight System on your computer

